

MasterClass in DSA

Exercise Set - 3

Topic: Singly Circular Linked List

(Exercises where solutions differ from Singly Linked List)

Total Exercises: 18

Easy: 5 | Intermediate: 8 | Difficult: 5

*CoreCode Programming Academy
An Academy by Yogeshwar Shukla*

General Instructions

1. This exercise set focuses on operations where Singly Circular Linked List (SCLL) solutions differ significantly from Singly Linked List (SLL).
2. Create a new header file `sclist.h` and implementation file `sclist.c` for these exercises.
3. In SCLL, the last node's next pointer points back to the first data node (not the head sentinel).
4. Key difference: Traversal termination checks if we've returned to the starting point, not NULL.
5. Use the same type definitions: `status_t`, `data_t`, `len_t`, `node_t` as in the SLL implementation.
6. Return appropriate status codes: `SUCCESS`, `LIST_EMPTY`, `LIST_DATA_NOT_FOUND`.
7. Test edge cases: empty list, single element, operations that affect circularity.

Key Structural Differences

SLL Termination: `while (p_run != NULL)`

SCLL Termination: `do { ... } while (p_run != p_first)` or track if returned to start

SLL Last Node: `p_last->next == NULL`

SCLL Last Node: `p_last->next == p_first_data_node`

List Notation: `[HEAD]->[d1]->[d2]->[d3]->⊙` (arrow loops back to d1)

Section A: Easy Exercises

Exercise 1: Create Circular List EASY

Write a function to create an empty singly circular linked list. Design the structure considering how you will handle the circular nature. Decide whether to use a head sentinel node and how to represent an empty circular list.

```
sclist_t* create_sclist(void)
```

Design Decision: Document your choice - will an empty list have head->next point to itself, or will it be NULL until the first element is added?

Expected Behavior:

Returns pointer to new empty circular list structure

Exercise 2: Insert at Start EASY

Write a function to insert a new node at the start of the circular list. Remember that the last node's next pointer must be updated to point to the new first node to maintain circularity.

```
status_t sclist_insert_start(sclist_t* p_list, data_t new_data)
```

Sample Input:

[HEAD]->[10]->[20]->[30]->⤿, new_data = 5

Sample Output:

[HEAD]->[5]->[10]->[20]->[30]->⤿ (30 now points to 5)

Exercise 3: Insert at End EASY

Write a function to insert a new node at the end of the circular list. The new node becomes the last node and must point back to the first data node.

```
status_t sclist_insert_end(sclist_t* p_list, data_t new_data)
```

Sample Input:

[HEAD]->[10]->[20]->[30]->⤿, new_data = 40

Sample Output:

[HEAD]->[10]->[20]->[30]->[40]->⤿ (40 points back to 10)

Exercise 4: Display Circular List EASY

Write a function to display all elements of the circular list. The function must correctly terminate after one complete traversal without infinite looping.

```
void sclist_show(sclist_t* p_list, const char* msg)
```

Sample Input:

[HEAD]->[10]->[20]->[30]->☺

Sample Output:

[START]->[10]->[20]->[30]->[CIRCULAR:10]

Exercise 5: Get Length EASY

Write a function that returns the number of data nodes in the circular list. Ensure correct termination after one complete traversal.

```
len_t sclist_get_length(sclist_t* p_list)
```

Sample Input:

[HEAD]->[10]->[20]->[30]->[40]->☺

Sample Output:

4

Section B: Intermediate Exercises

Exercise 6: Delete First Node INTERMEDIATE

Write a function to delete the first data node from the circular list. The last node's next pointer must be updated to point to the new first node. Handle the case when deleting the only node in the list.

```
status_t sclist_remove_start(sclist_t* p_list)
```

Sample Input:

[HEAD]->[10]->[20]->[30]->⌚

Sample Output:

[HEAD]->[20]->[30]->⌚ (30 now points to 20)

Exercise 7: Delete Last Node INTERMEDIATE

Write a function to delete the last node from the circular list. The second-to-last node becomes the new last node and must point back to the first data node.

```
status_t sclist_remove_end(sclist_t* p_list)
```

Sample Input:

[HEAD]->[10]->[20]->[30]->⌚

Sample Output:

[HEAD]->[10]->[20]->⌚ (20 now points to 10)

Exercise 8: Rotate List by K Positions INTERMEDIATE

Write a function to rotate the circular list by K positions. In a circular list, this is equivalent to changing where the "start" of the list is considered to be. This operation is more natural in circular lists than in linear lists.

```
status_t sclist_rotate(sclist_t* p_list, int k)
```

Sample Input:

[HEAD]->[1]->[2]->[3]->[4]->[5]->⌚, k = 2

Sample Output:

[HEAD]->[3]->[4]->[5]->[1]->[2]->⌚

Exercise 9: Split Circular List into Two Circular Lists

INTERMEDIATE

Write a function to split one circular list into two circular lists. Split at the middle - if odd number of nodes, first list gets the extra node. Both resulting lists must be properly circular.

```
sclist_t* sclist_split_half(sclist_t* p_list)
```

Sample Input:

[HEAD]->[1]->[2]->[3]->[4]->[5]->[6]->⌚

Sample Output:

Original: [HEAD]->[1]->[2]->[3]->⌚, New: [HEAD]->[4]->[5]->[6]->⌚

Exercise 10: Concatenate Two Circular Lists INTERMEDIATE

Write a function to concatenate two circular lists into one circular list. The second list's nodes are appended after the first list's last node. The result must be a single properly circular list. The second list becomes empty.

```
status_t sclist_concat(sclist_t* p_list_1, sclist_t** pp_list_2)
```

Sample Input:

List1: [HEAD]->[1]->[2]->⌚, List2: [HEAD]->[3]->[4]->⌚

Sample Output:

List1: [HEAD]->[1]->[2]->[3]->[4]->⌚, List2: empty

Exercise 11: Convert SLL to SCLL INTERMEDIATE

Write a function that takes a singly linked list (as implemented in Exercise Set 2) and converts it to a singly circular linked list by connecting the last node back to the first data node.

```
sclist_t* convert_sll_to_scll(list_t* p_sll)
```

Sample Input:

SLL: [HEAD]->[10]->[20]->[30]->[NULL]

Sample Output:

SCLL: [HEAD]->[10]->[20]->[30]->⌚

Exercise 12: Convert SCLL to SLL INTERMEDIATE

Write a function that takes a singly circular linked list and converts it to a regular singly linked list by breaking the circular link (last node points to NULL).

```
list_t* convert_scll_to_sll(sclist_t* p_scll)
```

Sample Input:

SCLL: [HEAD]->[10]->[20]->[30]->⌚

Sample Output:

SLL: [HEAD]->[10]->[20]->[30]->[NULL]

Exercise 13: Traverse N Times Around INTERMEDIATE

Write a function that traverses the circular list exactly N complete rounds and prints each element visited. This demonstrates the cyclic nature of the list.

```
void sclist_traverse_n_rounds(sclist_t* p_list, int n)
```

Sample Input:

[HEAD]->[1]->[2]->[3]->⌚, n = 2

Sample Output:

1 2 3 1 2 3

Section C: Difficult Exercises

Exercise 14: Josephus Problem DIFFICULT

Implement the classic Josephus Problem: N people stand in a circle, and every K-th person is eliminated until only one remains. Return the position (1-based) of the survivor. This is the quintessential circular list problem.

```
int josephus(int n, int k)
```

Sample Input:

`n = 7 (people), k = 3 (every 3rd eliminated)`

Sample Output:

`4 (person at position 4 survives)`

Elimination order for n=7, k=3: 3, 6, 2, 7, 5, 1, survivor: 4

Exercise 15: Josephus with Names DIFFICULT

Extend the Josephus Problem to work with a circular list of names (strings). Print the elimination order and return the survivor's name. Modify the node structure to hold string data.

```
char* josephus_with_names(sclist_str_t* p_list, int k)
```

Sample Input:

`Names: [Alice]->[Bob]->[Charlie]->[David]->⌚, k = 2`

Sample Output:

`Eliminated: Bob, David, Alice. Survivor: Charlie`

Exercise 16: Round Robin Scheduler DIFFICULT

Implement a round-robin CPU scheduler simulation using a circular list. Each node represents a process with a remaining burst time. Each round, the current process runs for a time quantum. If a process completes (burst time reaches 0), remove it. Continue until all processes complete. Print execution order.

```
void round_robin_schedule(sclist_process_t* p_list, int time_quantum)
```

Sample Input:

`Processes: [P1:5]->[P2:3]->[P3:7]->⌚, quantum = 2`

Sample Output:

P1(2) P2(2) P3(2) P1(2) P2(1-done) P3(2) P1(1-done) P3(2) P3(1-done)

Exercise 17: Merge Two Sorted Circular Lists DIFFICULT

Write a function to merge two sorted circular lists into a single sorted circular list. The resulting list must maintain sorted order and be properly circular.

```
sclist_t* sclist_merge_sorted(sclist_t* p_list_1, sclist_t* p_list_2)
```

Sample Input:

List1: [HEAD]->[1]->[3]->[5]->⌚, List2: [HEAD]->[2]->[4]->[6]->⌚

Sample Output:

[HEAD]->[1]->[2]->[3]->[4]->[5]->[6]->⌚

Exercise 18: Reverse Circular List DIFFICULT

Write a function to reverse a singly circular linked list in-place. After reversal, the list must remain circular with the last node (previously first) pointing to the new first node (previously last).

```
status_t sclist_reverse(sclist_t* p_list)
```

Sample Input:

[HEAD]->[1]->[2]->[3]->[4]->⌚

Sample Output:

[HEAD]->[4]->[3]->[2]->[1]->⌚ (1 points back to 4)

Hint: After reversing the links, you need to identify the new last node (original first) and ensure it points to the new first node (original last).

Summary: SLL vs SCLL Key Differences

Traversal Pattern

SLL:

```
for(p_run = p_list->next; p_run != NULL; p_run = p_run->next)
```

SCLL:

```
if (p_list->next != NULL) {  
    p_run = p_list->next;  
    do {  
        // process p_run  
        p_run = p_run->next;  
    } while (p_run != p_list->next);  
}
```

Operations with Significant Differences

Insert at Start: Must update last node's next to new first node

Insert at End: New node must point back to first data node

Delete First: Must update last node's next pointer

Delete Last: New last must point to first data node

Rotation: Simpler in SCLL - just change where head points

Split/Concat: Must maintain circularity in resulting lists

Reversal: Must reconnect to maintain circularity

SCLL-Specific Applications

Josephus Problem: Natural fit for circular elimination

Round Robin Scheduling: Processes cycle continuously

Circular Buffers: Fixed-size rotating storage

Game Turn Management: Players take turns in cycle

Playlist Repeat: Music loops back to start

Exercises from Set 2 That Transfer Directly

The following exercises from Exercise Set 2 can be solved with minimal changes (only traversal termination condition):

- Sum, Count, Min, Max, Average operations
- Search and membership checks
- Get data at index
- Count occurrences, count in range
- Check sorted, check equality
- Replace all occurrences

Students should practice adapting these as warm-up exercises.